

Registadores

32 regs de propósito geral (pode usar em qquer lugar)

\$zero - sempre tem o valor zero '0'

\$t0 - \$t7 - temporários

\$s0 - \$s7 - salvos

\$a0 - \$a3 - argumentos de funções

\$v0 e \$v1 - valor de retorno de uma função

\$ra - (return address) end. de retorno de uma função

Formatos das Instruções

três formatos

todas as instruções vão ser codificadas usando exatamente 32 bits.



(R)

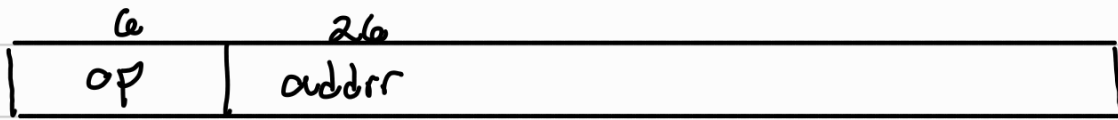
aprox. 90% de todas as instruções vão estar neste formato.



(I)

↑
imediato
constante

↑
regular



(J)
↑
jump

Instruções - Aritméticas

① add - soma o valor contido em dois registradores

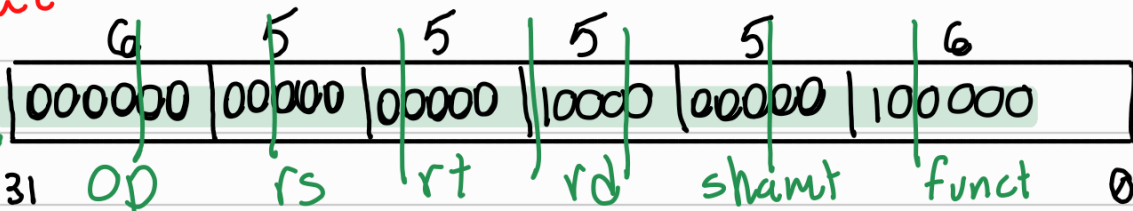
add \$rd, \$rs, \$rt
 $rd \leftarrow rs + rt$

sintaxe
semântica

Ex: ^{op+funct} add rd \$s0, ^{rs} \$zero, ^{rt} \$zero

$s0 \leftarrow 0 + 0$

cód. montagem (assembly)



cód. de máquina

(R)

0 0 0 0 8 0 2 0_H

② sub \$rd, \$rs, \$rt
 $rd \leftarrow rs - rt$

sintaxe
semântica

Ex: sub \$a0, \$s5, \$s7

③ mult \$rd, \$rs, \$rt
 $rd \leftarrow |rs \times rt|_{31-0}$

④ div \$rd, \$rs, \$rt
 $rd \leftarrow \text{mod}(rs, rt)$

← retorna só a parte inteira da divisão.
(ignora o resto)

⑤ `addi $rt, $rs, cte16` sintaxe
 $rt \leftarrow rs + cte$ semântica

Ex: `addi $s0, $zero, 42` # $s0 \leftarrow 0 + 42$

Instruções Lógicas

⑥ `and $rd, $rs, $rt` sint.
 $rd \leftarrow rs \& rt$

rs 0101
rt 1011 &
 0001

⑦ `or $rd, $rs, $rt`
 $rd \leftarrow rs | rt$

⑧ `xor $rd, $rs, $rt`
 $rd \leftarrow rs \oplus rt$

⑨ `nor $rd, $rs, $rt`
 $rd \leftarrow \overline{rs + rt}$

⑩ `andi $rt, $rs, cte`
 $rt \leftarrow rs \& cte$

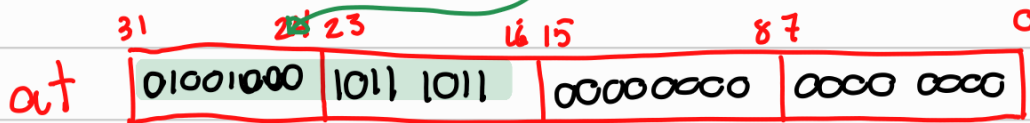
⑪ `ori $rt, $rs, cte`
 $rt \leftarrow rs | cte$

⑫ `xori $rt, $rs, cte`
 $rt \leftarrow rs \oplus cte$

⑬ lui - load upper immediate

Ex! Deseja-se colocar a cte 0x48BB|A935 no reg. \$s0.

lui \$at, 0x48BB



ori \$s0, \$at, 0xA935

at	4	8	B	B	0	0	0	0
cte	0	0	0	0	A	9	3	5
s0	4	8	B	B	A	9	3	5

⑭ sll - shift left logical multiplicação por potência de 2

at 0x00483376

sll \$at, \$at, 4 0x04833760

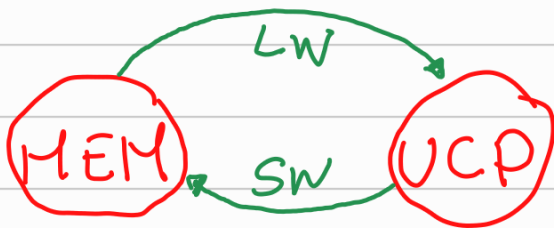
sll \$rd, \$rs, shamt (shift amount)

⑮ srl - shift right logical

srl \$rd, \$rs, shamt

16 lw / lb
17 sw / sb
18 beq 19.5 slt
19 bne
20 j
21 jr
22 jal

Acesso à Memória



UES

16.
Sintaxe: lw \$rt, cte(\$rs)

Semântica: $rt \leftarrow MEM[rs + cte]$

00000000	0B
00000001	49
00000002	35
3	2A
4	2A
5	00
6	05
7	19
8	1C
9	29
A	3F
B	5B
:	
:	

00 | 00 | C0 | 04 | \$S5

Ex: |w \$S0, 0(\$S5)

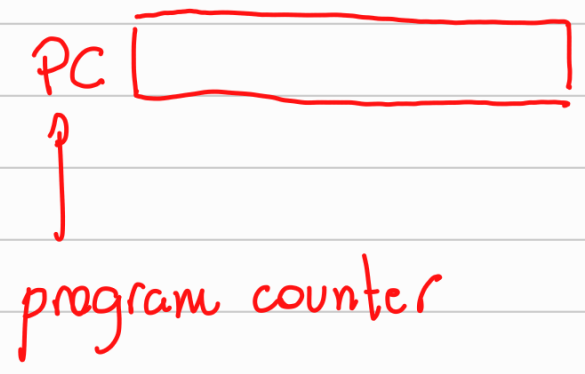
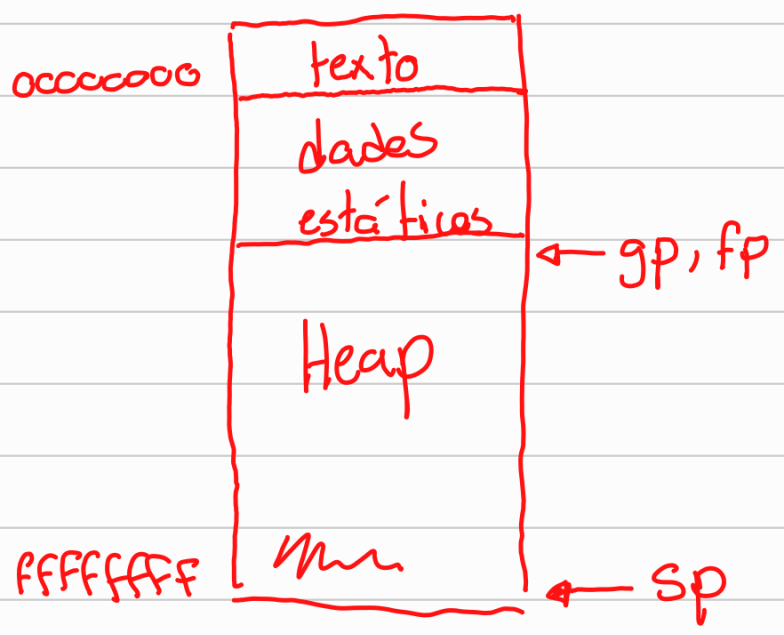
S0 ← MEM[S5+0]

19 | 05 | 00 | 2A | S0

17 sw \$rt, cte(\$rs)

MEM[rs+cte] ← rt

18. beq - branch if equal



Sintaxe: `beg $rs, $rt, Label`

semântica:

se $rs == rt$
 $PC = Label$

senão

$PC = PC + 4$

19 `bne` - branch if not equal

`bne $rs, $rt, Label`

se $rs \neq rt$
 $PC = Label$

senão

$PC = PC + 4$

20. `slt` - set if less than

`slt $rd, $rs, $rt`

semântica: se $rs < rt$
 $rd \leftarrow 1$

senão

$rd \leftarrow 0$

21. `j` - jump

$PC = PC[31:28] || Label:00$

sintaxe: `j Label`

semântica: $PC = Label$

22. jr jump register

jr \$rs

$PC = rs$

23: jal - jump and link

syntax: jal Label

sem: $\$ra \leftarrow PC + 4$
 $PC \leftarrow \text{Label}$